

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №2.

Тема: «Классы и объекты. Использование конструкторов»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

1. Определить пользовательский класс.
2. Определить в классе следующие конструкторы:
 - a. без параметров (конструктор по умолчанию)
 - b. с параметрами
 - c. копирования
3. Определить в классе деструктор.
4. Определить в классе компоненты-функции для просмотра и установки полей данных:
 - a. селекторы (get-методы для просмотра)
 - b. модификаторы (set-методы для установки)
5. Написать демонстрационную программу, в которой продемонстрировать:
 - a. все три случая вызова конструктора-копирования
 - b. вызов конструктора с параметрами
 - c. вызов конструктора без параметров

Пользовательский класс: АБИТУРИЕНТ (ABITURIENT)

Поля класса:

- ФИО абитуриента – string
- Специальность – string
- Балл ЕГЭ – int

Анализ задачи:

1. Создаем класс Abiturient с тремя полями (ФИО, специальность, балл ЕГЭ). Реализуем:
 - a. 3 конструктора: по умолчанию (пустые значения), с параметрами (передаем ФИО, специальность, балл), копирования (копируем объект)
 - b. Деструктор (освобождает ресурсы)
 - c. Get-методы (getFIO(), getSpecialty(), getScore()) для чтения полей
 - d. Set-методы (setFIO(), setSpecialty(), setScore()) для изменения полей
2. В функции main() создаем объекты разными способами и показываем, как работают все три конструктора.

Код:

```
#include <iostream>
```

```

#include <string>

using namespace std;

// КЛАСС ABITURIENT
class Abiturient {
private:
    // Поля класса (данные)
    string fio;           // ФИО абитуриента
    string specialty;     // Специальность
    int egeScore;         // Балл ЕГЭ

public:
    // КОНСТРУКТОРЫ

    // 1. Конструктор без параметров (по умолчанию)
    // Инициализирует поля значениями по умолчанию
    Abiturient() : fio("Не задано"), specialty("Не задана"), egeScore(0) {
        cout << "[Конструктор] Без параметров\n";
    }

    // 2. Конструктор с параметрами
    // Принимает ФИО, специальность и балл ЕГЭ
    Abiturient(string f, string s, int score) : fio(f), specialty(s), egeScore(score) {
        cout << "[Конструктор] С параметрами\n";
    }

    // 3. Конструктор копирования
    // Создаёт копию существующего объекта
    Abiturient(const Abiturient& other) : fio(other.fio), specialty(other.specialty),
    egeScore(other.egeScore) {
        cout << "[Конструктор] Копирования\n";
    }

    // ДЕКТРУКТОР
    // Вызывается автоматически при уничтожении объекта
    ~Abiturient() {
        cout << "[Деструктор] Вызван\n";
    }
}

```

```

// СЕЛЕКТОРЫ (get-методы)
// Методы для получения значений полей

string getFio() const {
    return fio;
}

string getSpecialty() const {
    return specialty;
}

int getEgeScore() const {
    return egeScore;
}

// МОДИФИКАТОРЫ (set-методы)
// Методы для изменения значений полей

void setFio(string f) {
    fio = f;
}

void setSpecialty(string s) {
    specialty = s;
}

void setEgeScore(int score) {
    egeScore = score;
}
};

// ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ

// Функция для демонстрации передачи объекта по значению
// При вызове этой функции срабатывает конструктор копирования
void passByValue(Abiturient obj) {
    // obj – копия переданного объекта

```

```

}

// Функция для демонстрации возврата объекта по значению
// При возврате срабатывает конструктор копирования
Abiturient returnByValue() {
    return Abiturient("Врем.", "Мат.", 50);
}

int main() {
    setlocale(LC_ALL, "Russian");

    // 1. Вызов конструктора без параметров
    cout << "1. Создание объекта (конструктор без параметров):\n";
    Abiturient obj1;
    cout << "\n";

    // 2. Вызов конструктора с параметрами
    cout << "2. Создание объекта (конструктор с параметрами):\n";
    Abiturient obj2("Иванов Иван", "Информатика", 85);
    cout << "\n";

    // 3. Случай 1: Прямая инициализация (конструктор копирования)
    cout << "3. Копирование объекта (случай 1: Abiturient obj3 = obj2):\n";
    Abiturient obj3 = obj2;
    cout << "\n";

    // 4. Случай 2: Передача объекта в функцию по значению
    cout << "4. Передача в функцию (случай 2: передача по значению):\n";
    passByValue(obj2);
    cout << "\n";

    // 5. Случай 3: Возврат объекта из функции по значению
    cout << "5. Возврат из функции (случай 3: возврат по значению):\n";
    Abiturient obj4 = returnByValue();
    cout << "\n";

    // 6. Демонстрация работы get/set методов
    cout << "6. Работа с get/set методами:\n";

```

```

    obj1.setFio("Петров Петр");
    obj1.setSpecialty("Экономика");
    obj1.setEgeScore(90);
    cout << "    ФИО: " << obj1.getFio() << endl;
    cout << "    Специальность: " << obj1.getSpecialty() << endl;
    cout << "    Балл ЕГЭ: " << obj1.getEgeScore() << endl;
    cout << "\n";

    return 0; // Успешное завершение программы
}

```

Результат выполнения:

```

1. Создание объекта (конструктор без параметров):
[Конструктор] Без параметров

2. Создание объекта (конструктор с параметрами):
[Конструктор] С параметрами

3. Копирование объекта (случай 1: Abiturient obj3 = obj2):
[Конструктор] Копирования

4. Передача в функцию (случай 2: передача по значению):
[Конструктор] Копирования
[Деструктор] Вызван

5. Возврат из функции (случай 3: возврат по значению):
[Конструктор] С параметрами

6. Работа с get/set методами:
    ФИО: Петров Петр
    Специальность: Экономика
    Балл ЕГЭ: 90

[Деструктор] Вызван
[Деструктор] Вызван
[Деструктор] Вызван
[Деструктор] Вызван

```

UML:

Abiturient
- fio: string - specialty: string - egeScore: int
+ Abiturient() + Abiturient(f: string, s: string, score: int) + Abiturient(other: const Abiturient&) + ~Abiturient() + getFio(): string + getSpecialty(): string + getEgeScore(): int + setFio(f: string): void + setSpecialty(s: string): void + setEgeScore(score: int): void

Вопросы:

1. Инициализация объектов при создании.
2. Три: без параметров, с параметрами, копирования.
3. Освобождение ресурсов. Явно — когда есть динамическая память или ресурсы.
4.
 - a. Без параметров — создание объекта с значениями по умолчанию
 - b. С параметрами — инициализация конкретными значениями
 - c. Копирования — создание копии объекта
5. При инициализации одного объекта другим, передаче по значению, возврате из функции.
6. Имя совпадает с классом, нет возвращаемого типа, вызывается автоматически при создании.
7. Имя ~класс(), нет параметров и возвращаемого типа, вызывается автоматически при уничтожении.
8. Ко всем (private, protected, public).
9. Указатель на текущий объект.
10. Внутри — inline (встраиваемые), вне — обычная компиляция.
11. Ничего (не возвращает).
12. Конструктор по умолчанию, деструктор, конструктор копирования.
13. Ничего (не возвращает).

- 14.Конструктор по умолчанию (без параметров).
- 15.Конструктор без параметров.
- 16.Конструктор с параметрами (string, int).
- 17.s1 — конструктор с параметрами, s2 — конструктор копирования.
- 18.s1 — конструктор с параметрами, s2 — конструктор без параметров,
оператор присваивания.
- 19.Конструктор копирования.
- 20.p.set_name("новое имя");